

TRABALHO FINAL

Curso de Python para Engenharia de Dados e IA

Pipeline de Dados Governamentais do Estado do Ceará:

ETL, Data Warehouse, Hadoop e Inteligência Artificial

Versão 1.0 | Junho de 2026

Sumário

Sumário.....	1
1. Contexto e Justificativa.....	1
2. Fontes de Dados.....	1
2.1 Banco de Dados PostgreSQL.....	1
2.2 API REST — Ceará Transparente (Contratos)	1
3. Problema de Negócio	1
3.1 Escopo do Trabalho Final	1
4. Arquitetura do Pipeline	1
4.1 Visão Geral.....	1
4.2 Fluxo 1 — Pipeline ETL com Airflow	1
DAG 1: Extração e Carga Bronze	1
DAG 2: Transformação Silver	1
DAG 3: Carga Gold — Data Warehouse	1
4.3 Fluxo 2 — Machine Learning e IA	1
Modelo 1: Detecção de Anomalias Contratuais.....	1
Modelo 2: Previsão de Pagamentos Trimestrais	1
Componente IA Generativa (Opcional Avançado).....	1
5. Requisitos Técnicos.....	1
5.1 Infraestrutura Mínima.....	1
5.2 Bibliotecas Python Necessárias	1
6. Entregas Esperadas	1
6.1 Artefatos Técnicos	1
6.2 Documentação.....	1
6.3 Critérios de Avaliação	1
7. Cronograma Sugerido	1
8. Dicas e Boas Práticas.....	1
8.1 Extração da API	1
8.2 Modelagem do DW	1
8.3 Machine Learning.....	1
8.4 Airflow.....	1
9. Estrutura de Diretórios Recomendada	1
10. Referências e Recursos	1
10.1 Documentação Oficial.....	1
10.2 Fontes de Dados	1
10.3 Artigos e Tutoriais Recomendados	1

1. Contexto e Justificativa

O Estado do Ceará disponibiliza dados públicos sobre contratos, empenhos e execução orçamentária por meio do portal Ceará Transparente. Esses dados possuem grande valor analítico para órgãos de controle, pesquisadores e cidadãos. Contudo, sua dispersão entre uma API REST e um banco de dados relacional PostgreSQL impede análises integradas e a extração de inteligência por meio de modelos de aprendizado de máquina.

Este trabalho final propõe o desenvolvimento de um pipeline de dados completo que integre essas duas fontes, aplique transformações de qualidade, armazene os resultados em um Data Warehouse (DW) e em um sistema de armazenamento distribuído (Hadoop/HDFS), e, por fim, utilize os dados consolidados para a construção de modelos de Machine Learning e Inteligência Artificial orientados ao monitoramento da execução contratual pública.

Objetivos Gerais

- (1) Integrar dados de contratos públicos a partir de API REST e banco PostgreSQL;
- (2) Construir um pipeline ETL orquestrado pelo Apache Airflow;
- (3) Persistir dados em camadas DW e Hadoop/HDFS;
- (4) Gerar inteligência via modelos de ML/IA sobre irregularidades e previsões contratuais.

2. Fontes de Dados

2.1 Banco de Dados PostgreSQL

O banco de dados relacional fornecido contém tabelas que registram a execução orçamentária do Estado do Ceará. A estrutura cobre desde a classificação funcional-programática até os documentos de empenho e ordem bancária.

Tabela	Chave Primária	Descrição
acao	id	Ações programáticas com vínculo a programas, órgãos, funções e subfunções
categoria	id	Categorias econômicas da despesa por ano de referência
elemento	id	Elementos de despesa detalhando a natureza do gasto
empenhos	(id, ano)	Documentos de empenho: credor, valor, data, classificação e itens
fonte_recurso	id	Fontes de financiamento das despesas públicas
funcao	id	Funções orçamentárias (saúde, educação, segurança etc.)
grupo	id	Grupos de natureza de despesa (pessoal, investimentos etc.)
natureza	id	Natureza detalhada da despesa
ordem_bancaria_orcamentaria	(id, ano)	Ordens bancárias: pagamento efetivo ao credor com dados bancários
pedidos	id	Requisições em formato JSONB (flexível)
unidade_gestora	(codigo, ano)	Unidades gestoras: CNPJ, tipo, órgão e esfera de governo

2.2 API REST — Ceará Transparente (Contratos)

A API pública do portal Ceará Transparente disponibiliza dados de contratos celebrados pelos órgãos estaduais. O endpoint principal utilizado no projeto é:

Endpoint Base

GET <https://api-dados-abertos.cearatr transparente.ce.gov.br/transparencia/contratos/contratos> Parâmetros: page, dataassinatura_inicio, dataassinatura_fim Paginação: até 100 registros por página (campo summary.total_pages indica o total)

Os principais campos retornados pela API incluem:

- Identificação: id, isn_sic, num_spu, num_contrato, cod_concedente, cod_gestora

- Partes: isn_parte_origem, isn_parte_destino, cpf_cnpj_financiador, descricao_nome_credor
- Valores: valor_contrato, valor_original_concedente, valor_atualizado_concedente, calculated_valor_pago
- Datas: dataassinatura, data_inicio, data_termino, data_publicacao_portal, data_rescisao
- Classificação: descricao_modalidade, descricao_tipo, tipo_objeto, descricao_objeto
- Status: descricao_situacao, infringement_status, accountability_status, emergency

3. Problema de Negócio

O Estado do Ceará celebra centenas de contratos anualmente, totalizando bilhões de reais. A fragmentação das informações entre o sistema orçamentário (PostgreSQL) e o portal de transparência (API) dificulta três questões críticas:

Questão 1 — Rastreabilidade Financeira

Dado um contrato público assinado, qual o percentual do valor contratado já foi efetivamente pago (ordens bancárias) e qual ainda está apenas empenhado? Há contratos com pagamentos acima do valor contratado?

Questão 2 — Detecção de Anomalias

Existem padrões anormais de contratação? Contratos de alto valor sendo celebrados com dispensa ou inexigibilidade de licitação, ou credores com histórico de infrações (infringement status > 0) recebendo novos contratos?

Questão 3 — Previsão e Planejamento

Com base no histórico de contratos e empenhos, é possível prever o volume de pagamentos no próximo trimestre por órgão? Quais contratos têm maior risco de não execução ou rescisão?

3.1 Escopo do Trabalho Final

O aluno deverá construir um sistema de dados end-to-end que responda às três questões acima. O sistema deve ser capaz de:

1. Extrair dados do PostgreSQL e da API REST de forma incremental (por data)
2. Transformar e unificar os dados em um modelo dimensional (Data Warehouse estrela ou floco de neve)
3. Armazenar dados brutos e transformados no HDFS (Hadoop)
4. Orquestrar todo o pipeline via Apache Airflow com DAGs agendadas
5. Treinar modelos de ML para detecção de anomalias e previsão de pagamentos
6. Gerar um relatório automático com os insights obtidos pelos modelos

4. Arquitetura do Pipeline

4.1 Visão Geral

A arquitetura segue o padrão Medallion (Bronze → Silver → Gold), adaptado para o contexto acadêmico com camadas claramente separadas por responsabilidade:

Camada	Tecnologia	Descrição
Bronze (Raw)	HDFS / PostgreSQL	Dados brutos extraídos sem transformação. Preservação total da fonte.
Silver (Staging)	HDFS (Parquet)	Dados limpos: deduplicados, tipados, campos nulos tratados, chaves padronizadas.
Gold (DW)	PostgreSQL (DW)	Modelo dimensional otimizado para consultas analíticas e alimentação de ML.
ML/IA	Python (scikit-learn, MLlib)	Modelos treinados e inferidos sobre a camada Gold.

4.2 Fluxo 1 — Pipeline ETL com Airflow

DAG 1: Extração e Carga Bronze

Responsável pela extração incremental dos dados brutos das duas fontes:

- Task `extract_postgres`: Conecta ao PostgreSQL, lê tabelas `empenhos`, `ordem_bancaria_orcamentaria` e `unidade_gestora` filtradas por `data`, salva como JSON/Parquet no HDFS (camada Bronze).
- Task `extract_api`: Realiza chamadas paginadas à API de contratos, respeitando o campo `total_pages` do `summary`. Salva cada página no HDFS.
- Task `validate_bronze`: Valida schema e completude mínima dos arquivos ingeridos (número de registros esperado, colunas obrigatórias presentes).

DAG 2: Transformação Silver

Aplica regras de qualidade e padronização sobre a camada Bronze:

- Conversão de tipos: datas ISO 8601 para timestamp, valores monetários para Decimal, CPF/CNPJ normalizados.
- Deduplicidade: contratos e empenhos com mesma chave natural são deduplicados mantendo a versão mais recente (campo `updated_at`).
- Enriquecimento: junção de empenhos com `unidade_gestora` para obter nome e CNPJ do órgão por exemplo.
- Persistência: Parquet particionado por ano/mês no HDFS, camada Silver.

DAG 3: Carga Gold — Data Warehouse

Popula o modelo dimensional no PostgreSQL DW

- `dim_credor`: CNPJ, nome, tipo (PJ/PF).
- `dim_orgao`: código, nome, CNPJ, tipo administração, esfera.
- `dim_modalidade`: tipo de licitação (pregão eletrônico, dispensa, inexigibilidade etc.).
- `dim_tempo`: data completa com atributos de ano, trimestre, mês, dia da semana.

- `fato_contrato`: chaves das dimensões + valor contratado, pago, empenhado, status, flag de emergência.
- `fato_empenho`: chaves + valor, data, modalidade, ligação com `fato_contrato` via `cod_contrato`.

4.3 Fluxo 2 — Machine Learning e IA

Modelo 1: Detecção de Anomalias Contratuais

Objetivo: identificar contratos com comportamento atípico em relação a valor, modalidade, credor e órgão contratante.

- Algoritmo sugerido: Isolation Forest ou Autoencoder (não supervisionado).
- Features: `valor_contrato`, modalidade (one-hot), `tipo_objeto`, `dias_vigencia`, `flag_emergency`, `historico_credor_infringement`.
- Output: score de anomalia (0 a 1) por contrato, armazenado na `fato_contrato`.

Modelo 2: Previsão de Pagamentos Trimestrais

Objetivo: prever o volume total de ordens bancárias por órgão para o próximo trimestre.

- Algoritmo sugerido: XGBoost Regressor ou Prophet (série temporal).
- Features: histórico trimestral de pagamentos, valor contratado ativo, tipo de despesa, ano eleitoral.
- Output: previsão de valor (R\$) com intervalo de confiança.

Componente IA Generativa

Utilizar um LLM (ex: API OpenAI ou modelo open-source via Ollama) para gerar automaticamente um relatório narrativo com os insights dos modelos, em linguagem acessível para gestores públicos sem formação técnica.

5. Requisitos Técnicos

5.1 Infraestrutura Mínima

Componente	Tecnologia	Finalidade
Orquestração	Apache Airflow 2.x	Agendamento e monitoramento das DAGs
Banco Origem	PostgreSQL 15+	Fonte: tabelas operacionais
API	Ceará Transparente REST	Fonte: contratos públicos
Armazenamento Distribuído	Hadoop HDFS 3.x	Camadas Bronze e Silver (Parquet)
Data Warehouse	PostgreSQL (DW)	Camada Gold — modelo dimensional
Processamento	Pandas	Transformações ETL nas DAGs
ML/IA	scikit-learn, XGBoost, MLlib	Treinamento e inferência dos modelos
Containerização	Docker / Docker Compose	Ambiente reproduzível para todo o stack
Linguagem	Python 3.10+	Toda a implementação do pipeline e modelos

5.2 Bibliotecas Python Necessárias

- airflow[postgres,http]: Operadores e conexões Airflow
- psycopg2-binary ou sqlalchemy: Conexão PostgreSQL
- requests / httpx: Chamadas à API REST
- pandas / pyarrow: Manipulação e serialização Parquet
- scikit-learn: Isolation Forest, pipelines de ML
- xgboost / prophet: Modelos de regressão e série temporal
- great_expectations (opcional): Validação de qualidade de dados

6. Entregas Esperadas

6.1 Artefatos Técnicos

7. Repositório Git com estrutura de projeto organizada (src/, dags/, models/, tests/, docker-compose.yml).
8. docker-compose.yml funcional que suba: Airflow, PostgreSQL (origem e DW), Hadoop (NameNode + DataNode) e Jupyter.
9. Mínimo 3 DAGs no Airflow: extração Bronze, transformação Silver e carga Gold.
10. Script de criação do schema do DW (DDL) com todas as tabelas dimensionais e de fato.
11. Notebook Jupyter demonstrando o treinamento e avaliação dos modelos de ML.
12. Script de inferência integrado ao Airflow (DAG de ML) que lê do DW, aplica o modelo e grava o resultado.
13. Apresentação Final dos Resultados

6.2 Documentação

- README.md com instruções de execução passo a passo.
- Diagrama da arquitetura do pipeline (imagem ou draw.io).
- Dicionário de dados do DW (tabelas, colunas, tipos, descrições).
- Relatório final (PDF e/ou Painel) com análise dos resultados dos modelos de ML.

7. Dicas e Boas Práticas

7.1 Extração da API

- Sempre inspecione o campo `summary.total_pages` antes de iterar as páginas para evitar laços infinitos.
- Armazene o último `data_assinatura` processado em uma variável Airflow (Variable) para extração incremental.
- Respeite o `rate limit` da API: adicione `sleep` entre chamadas de páginas quando o volume for alto.

7.2 Modelagem do DW

- Use surrogate keys (SERIAL ou BIGSERIAL) nas dimensões, nunca as chaves de negócio como PK do DW.
- Aplique Slowly Changing Dimension tipo 2 (SCD2) em `dim_credor` para rastrear mudanças de razão social ao longo do tempo.
- Particionamento da `fato_contrato` por ano garante queries mais rápidas em consultas históricas.

7.3 Machine Learning

- Trate o desbalanceamento: contratos anômalos são minoria. Use SMOTE ou ajuste de pesos caso converta para classificação supervisionada.
- Registre métricas e artefatos de cada experimento com MLflow para rastreabilidade.
- Avalie o modelo de anomalia com analistas de controle, pois não há rótulos ground-truth pré-definidos.

7.4 Airflow

- Defina `depends_on_past=False` e use `catchup=False` para novas DAGs, a menos que a carga histórica seja intencional.
- Separe operações de extração, transformação e carga em tasks distintas para melhor observabilidade e retentativa granular.
- Use XCom apenas para metadados leves (caminhos HDFS, contagens); nunca transfira DataFrames inteiros via XCom.

8. Estrutura de Diretórios Recomendada

```
projeto-final/ ├── dags/ |   ├── dag_bronze_extract.py |   ├── dag_silver_transform.py
|   ├── dag_gold_load.py |   ├── dag_ml_inference.py |   ├── src/ |   ├── extractors/ |
|   ├── postgres_extractor.py |   ├── api_extractor.py |   ├── transformers/ |
|   ├── bronze_to_silver.py |   ├── silver_to_gold.py |   ├── loaders/ |
dw_loader.py ├── models/ |   ├── anomaly_detection.py |   ├── payment_forecast.py
|   ├── notebooks/ |   ├── eda_e_treinamento_ml.ipynb |   ├── sql/ |   ├── dw_schema.sql
|   ├── tests/ |   ├── test_extractors.py |   ├── docker-compose.yml |   ├── requirements.txt
|   └── README.md
```

10. Referências e Recursos

10.1 Documentação Oficial

- Apache Airflow: <https://airflow.apache.org/docs/>
- Apache Hadoop HDFS: <https://hadoop.apache.org/docs/>
- scikit-learn: <https://scikit-learn.org/stable/>
- XGBoost: <https://xgboost.readthedocs.io/>
- Prophet (Meta): <https://facebook.github.io/prophet/>
- PySpark MLlib: <https://spark.apache.org/docs/latest/ml-guide.html>

10.2 Fontes de Dados

- Portal Ceará Transparente: <https://cearatransparente.ce.gov.br/>
- API de Dados Abertos: <https://api-dados-abertos.cearatransparente.ce.gov.br/>

10.3 Artigos e Tutoriais Recomendados

- Databricks. "Medallion Architecture." databricks.com/glossary/medallion-architecture
- Kimball, R.; Ross, M. "The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling." 3rd ed. Wiley, 2013.
- Liu, F. T. et al. "Isolation Forest." IEEE ICDM, 2008.

Proposta elaborada como guia para o Trabalho Final do Curso de Python para Engenharia de Dados e IA — 2026.